Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

AUGUST/RESIT EXAMINATIONS 2022/2023

MODULE: CA320 - Computability and Complexity

PROGRAMME(S):

CASE	BSc in Computer Applications (Sft.Eng.)
ECSAO	Study Abroad (Engineering & Computing)

YEAR OF STUDY: 3, O

EXAMINERS: Dr. David Sinclair (Ext. 5510)

TIME ALLOWED: 2 hours

INSTRUCTIONS: Answer 4 Questions.
All questions carry equal marks.

PLEASE DO NOT TURN OVER THIS PAGE UNTIL INSTRUCTED TO DO SO

The use of programmable or text storing calculators is expressly forbidden.
Please note that where a candidate answers more than the required number of questions, the examiner will mark all questions attempted and then select the highest scoring ones.

Requirements for this paper (Please mark (X) as appropriate)

☐	*Log Tables*	☐	*Thermodynamic Tables*
☐	*Graph Paper*	☐	*Actuarial Tables*
☐	*Dictionaries*	☐	*MCQ Only - Do not publish*
☐	*Statistical Tables*	☐	*Attached Answer Sheet*

Section A

QUESTION 1 ***[Total marks: 25]***

1(a) [5 Marks]

Describe *list comprehensions* in the Haskell Programming language. Illustrate your answer with exampels.

1(b) [8 Marks]

Write a Haskell function called `vowels` that take a string and *using list comprehension* returns a string containing all the vowels from the original string. Include the function type for `vowels`. For example, `vowels "Hello World"` would evaluate to `"eoo"`.

1(c) [12 Marks]

Without using the list concatenation operator $++$, write a Haskell function that takes a list of lists as input and returns a list that is the concatenation of all the original lists. For example, if the input to the function was `[[3,2,1],[8,9],[1,2,3,4]]`, the function would evaluate to `[3,2,1,8,9,1,2,3,4]`.

[End Question 1]

QUESTION 2 ***[Total marks: 25]***

2(a) [5 Marks]

Using examples, show how the `let` and `where` constructs are used in Haskell.

2(b) [5 Marks]

Using an example, illustrate how the `case` statement can be used in conjunction with pattern matching in Haskell.

2(c) [15 Marks]

Representing sets as lists, write Haskell functions that implements set union and set difference. Remember sets do not have duplicates. The function should use class constraints to ensure that it will work for sets of integers, floats, and characters.

[End Question 2]

Section B

QUESTION 3 ***[Total marks: 25]***

3(a) [5 Marks]

Describe the complexity classes $\mathbf{P}$ and $\mathbf{NP}$ and what is the relationship between them?

3(b) [10 Marks]

Prove that a language L belongs to $\mathbf{NP}$ if and only if a language $L_{check} \in \mathbf{P}$ and a polynomial p exist such that

$$L = \{x | \exists y . x, y \in L_{check} \wedge y \leq p(x)\}$$

3(c) [10 Marks]

Describe a problem that belongs to the class $\mathbf{NP}$ and, using the theorem in part(b) of this question, prove that it is a $\mathbf{NP}$ problem.

[End Question 3]

QUESTION 4 ***[Total marks: 25]***

4(a) [5 Marks]

Formally define a *context-free grammar*.

4(b) [10 Marks]

Formally describe the simplest automaton that can recognise a context-free grammar.

4(c) [10 Marks]

Construct an automaton (of the type indicated in part (b)) that recognises the language $L = \{ww^r | w^r$ is the reverse of the string $w\}$ over the alphabet $\Sigma = \{a, b, c\}$.

[End Question 4]

QUESTION 5 ***[Total marks: 25]***

5(a) [5 Marks]

In your own words briefly describe the difference between *computability* and *complexity*.

5(b) [20 Marks]

In the entrance to the School of Computing building there us a sign that say:

Redefining "Possible" with Computing

In the context of Computability, critique this phrase. Your answer should address the term *possible* in the context of Computability; and how could future technologies impact this phrase?

[End Question 5]

[END OF EXAM]